PROJET : SYSTEME DE SUPERVISION DISTRIBUE

COURS : SYSTEME REPARTI

ENSEIGNANT : MR MAURICE Djibril Faye

Réaliser par :

. KHADY DITE SOPHIE NDIAYE

. NDEYE NDACKETE SECK

.OMAR GUINDO

ANNEE UNIVERSITAIRE : 2023/2024

UNIVERSITE : UNIVERSITE NUMERIQUE CHEIKH AHMIDOU KANE

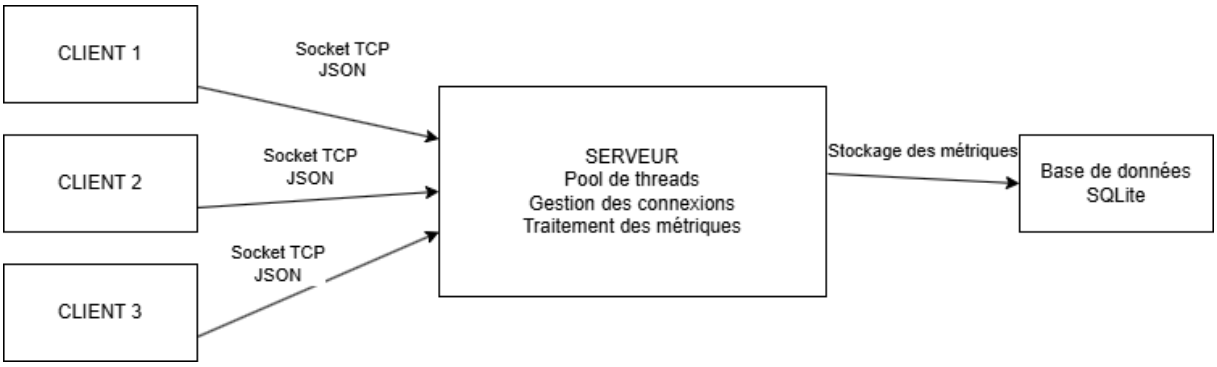
INTRODUCTION :

Dans les environnements informatiques modernes, la supervision des systèmes est essentielle afin d'assurer la disponibilité et les performances des infrastructures.

L'objectif de ce projet est de concevoir et de mettre en œuvre un système de supervision distribué basé sur une architecture client-serveur. Chaque machine supervisée exécute un agent chargé de collecter les métriques systèmes telles que l'utilisation du CPU, la mémoire, l'espace disque et l'état des services.

Ces informations sont envoyées périodiquement à un serveur central qui les analyse et les stocke dans une base de données afin de permettre leur consultation et la détection d'anomalies.

ARCHITECTURE DU SYSTEME



L'architecture du système repose sur un modèle client-serveur.

Chaque machine supervisée exécute un agent de supervision (client) chargé de collecter les métriques systèmes tels que l'utilisation CPU, la mémoire, le disque, l'uptime, le système d'exploitation et l'état des services.

Ces informations sont envoyées périodiquement au serveur central via une connexion socket TCP.

Le serveur de supervision est multi-clients et utilise les threads pour gérer plusieurs connexions simultanément. Les métriques reçues sont analysées puis stockées dans une base de données SQLite afin de permettre leur consultation et leur exploitation.

LES CHOIX TECHNIQUES

Plusieurs choix techniques ont été réalisés pour la conception et l'implémentation du système de supervision.

Le langage Python a été choisi en raison de sa simplicité, de sa lisibilité et de la richesse de ses bibliothèques pour la programmation réseau et la gestion des systèmes.

La communication entre les clients et le server repose sur les sockets TCP, qui permettent une communication fiable entre les différentes machines du système distribué.

Le format JSON a été utilisé pour structurer les messages échangés entre les clients et le server. Ce format est léger, lisible et facilement manipulable en Python.

Pour le stockage des métriques collectées, nous avons utilisé SQLite, une base de données légère intégrée directement dans Python est adaptée aux projets de petite à moyenne taille.

Enfin, la gestion de plusieurs clients simultanément est assurée grâce à l'utilisation de multi-threading, permettant au server de traiter plusieurs connexions en parallèle.

PROTOCOLE DE COMMUNICATION

La communication entre l'agent de supervision (client) et le serveur central repose sur une connexion réseau utilisant le protocole TCP via des sockets.

Les données échangées utilisent un protocole applicatif basé sur le format JSON. Ce format permet de structurer les informations envoyées par le client de manière claire et facilement exploitable par le serveur.

Chaque agent de supervision collecte périodiquement les métriques systèmes puis les envoie au serveur sous forme de message JSON contenant les informations suivantes :

. L'utilisation du CPU

. L'utilisation de la mémoire

. L'utilisation du disque

. L'uptime du système

. Le système d'exploitation

. Le type de processeur

. L'état des services surveillés

. L'horodatage du message

Le serveur reçoit ces messages via un socket TCP, les décode à l'aide du module JSON puis valide les données reçues avant de les enregistrer dans la base de données SQLite.

L'utilisation du format JSON facilite l'échange de données entre les différents composants du système et assure une bonne extensibilité du protocole.

LA GESTION DE LA CONCURRENCE

Dans un système de supervision distribué, plusieurs clients peuvent envoyés métriques au serveur en même temps. Il est donc nécessaire de mettre en place un mécanisme permettant de gérer plusieurs connexions simultanées.

Pour résoudre ce problème, le serveur utilise une gestion de concurrence basée sur les threads. Chaque fois qu'un se connecte, le serveur crée un thread chargé de traiter les messages envoyés par ce client.

Cette approche permet au serveur de continuer à accepter de nouvelles connexions pendant que les autres clients sont déjà en cours de traitement.

Dans notre implémentation, un pool de threads est utiliser afin d'optimiser la gestion des ressources du serveur. Au lieu de créer un nouveau thread à chaque connexion, un ensemble de threads est maintenu actif et réutilisé pour traiter les requêtes des clients.

Cette méthode présente plusieurs avantages :

. Elle permet de gérer plusieurs clients simultanément

. Elle réduit le cout de création et destruction des threads

. Elle améliore la performance globale du serveur

COMPARAISON DES APPROCHES POSSIBLES

Plusieurs stratégies peuvent être utilisées pour gérer la concurrence dans un serveur réseau :

1) Serveur séquentiel

Dans ce modèle, le serveur traite un client à la fois. Cette solution est simple à implémenter mais devient rapidement inefficace lorsque plusieurs clients se connectent simultanément.

2) Création d'un thread par client

Chaque connexion client est traitée dans un thread indépendant. Cette approche permet le traitement parallèle mais peut devenir couteuse si le nombre de clients augmente.

3) Pool de threads (solution choisie)

Le pool de threads consiste à maintenir un nombre limité de threads disponible pour traiter les requêtes des clients. Les connexions entrantes sont distribuées entre ces threads.

Cette solution offre un bon compromis entre performances, gestion des ressources et scalabilité, ce qui explique son choix dans ce projet.

Conclusion sur la concurrence

L'utilisation d'un pool de threads permet au serveur de superviser efficacement plusieurs clients simultanément tout en maintenant une utilisation raisonnable des ressources système.

RESULTATS EXPERIMENTAUX

Afin d'évaluer les performances du système de supervision, plusieurs tests de charges ont été réalisés. Ces testent consistent à simuler plusieurs clients envoyant simultanément leurs métriques au serveur.

L'objectif est de vérifier la capacité du serveur à gérer plusieurs connexions en parallèle tout en continuant à recevoir et enregistrer les données correctement.

Test avec 10 clients

Dans ce premier test, 10 clients envoient périodiquement leurs métriques systèmes au serveur.

Résultats observés :

. Le serveur accepte toutes les connexions

. Les métriques sont correctement reçues

. Les données sont enregistrées dans la base SQLite

. Aucune perte de message n'est observée

La gestion multi-thread permet au serveur de traiter les clients simultanément sans ralentissement notable.

Test avec 50 clients

Dans ce test, le nombre de clients est augmenté 50 connexions simultanées.

Résultats observés :

. Le serveur continue à recevoir les métriques correctement

. Les threads permettent de traiter plusieurs connexions en parallèle

. Une légère augmentation du temps de traitement peut être observée

. La base de données continue à stocker les informations sans erreur

Ces résultats montrent que l'architecture du système reste stable même avec un nombre plus important de clients.

Test avec 100 clients

Le dernier test consiste à simuler 100 clients envoyant leurs métriques au serveur.

Résultats observés :

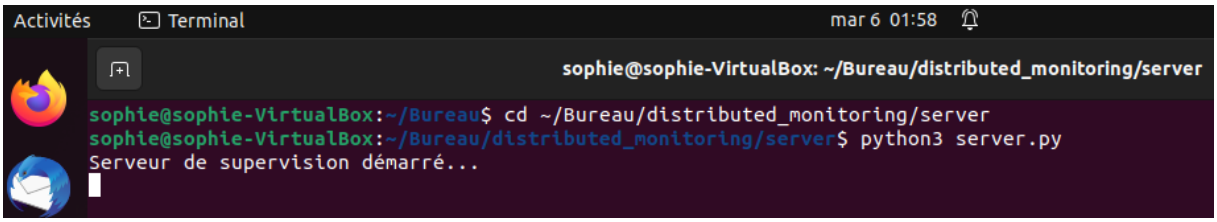
.Le serveur parvient à gérer les connexions simultanées

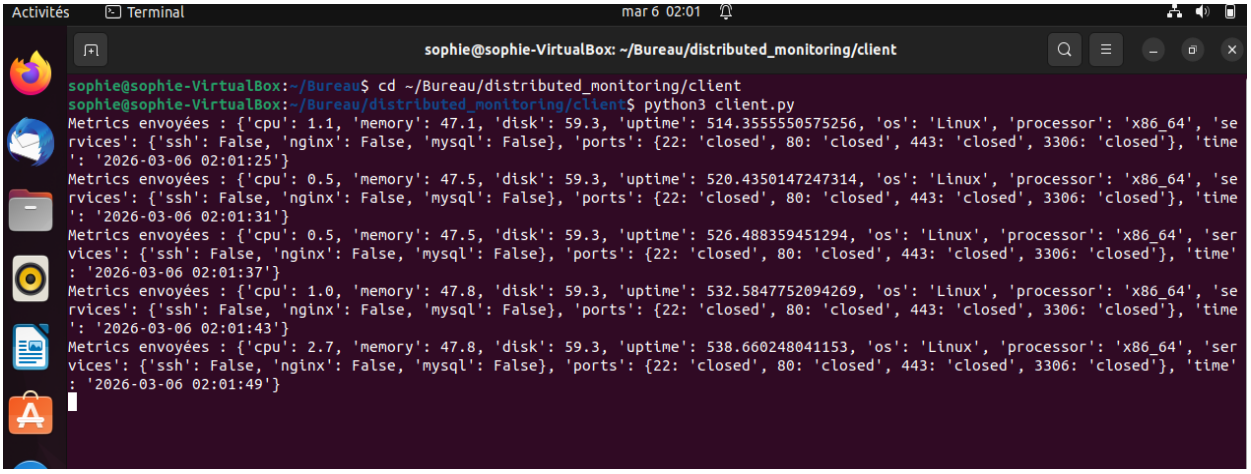
. Les métriques sont toujours reçues et traitées

. L'utilisation CPU du serveur augmente

. Les performances restent acceptables grâce à l'utilisation du pool threads

Ce test démontre que le système peut être utilisé dans un environnement comportant un nombre important de machines supervisées.





```
Activités  Terminal                                   mar 6  02:04
                    sophie@sophie-VirtualBox: ~/Bureau/distributed_monitoring/server
sophie@sophie-VirtualBox:~/Bureau$ cd ~/Bureau/distributed_monitoring/server
sophie@sophie-VirtualBox:~/Bureau/distributed_monitoring/server$ python3 server.py
Serveur de supervision démarré...
Client connecté : ('127.0.0.1', 38300)

--- Nouvelle métrique reçue ---
Client : ('127.0.0.1', 38300)
CPU : 1.1 %
Mémoire : 47.1 %
Disque : 59.3 %
Uptime : 514.3555550575256
Timestamp : 2026-03-06 02:01:25
OS : Linux
Processor : x86_64
Services : {'ssh': False, 'nginx': False, 'mysql': False}
Ports : {'22': 'closed', '80': 'closed', '443': 'closed', '3306': 'closed'}
```

```
s  Terminal                                           mar 6  15:04
                    sophie@sophie-VirtualBox: ~/Bureau/distributed_monitor
Client connecté : ('127.0.0.1', 38940)

--- Nouvelle métrique reçue ---
Client : ('127.0.0.1', 38940)
CPU : 100.0 %
ALERTE : CPU supérieure à 90%
Mémoire : 48.2 %
Disque : 59.4 %
Uptime : 1112.7135903835297
Timestamp : 2026-03-06 15:00:03
OS : Linux
Processor : x86_64
Services : {'ssh': False, 'nginx': False, 'mysql': False}
Ports : {'22': 'closed', '80': 'closed', '443': 'closed', '3306': 'closed'}
```

```
sqlite> SELECT * FROM metrics;
1|('127.0.0.1', 59350)|1.6|46.2|59.3|1088.67881393433|||||2026-03-06 00:49:44
2|('127.0.0.1', 39610)|17.7|46.6|59.3|1094.77935886383|||||2026-03-06 00:49:50
3|('127.0.0.1', 39622)|0.5|46.6|59.3|1100.82681369781|||||2026-03-06 00:49:56
4|('127.0.0.1', 38300)|1.1|47.1|59.3|514.355555057526|||||2026-03-06 02:01:25
5|('127.0.0.1', 38306)|0.5|47.5|59.3|520.435014724731|||||2026-03-06 02:01:31
6|('127.0.0.1', 38306)|0.5|47.5|59.3|526.488359451294|||||2026-03-06 02:01:37
7|('127.0.0.1', 38308)|1.0|47.8|59.3|532.584775209427|||||2026-03-06 02:01:43
8|('127.0.0.1', 59114)|2.7|47.8|59.3|538.660248041153|||||2026-03-06 02:01:49
9|('127.0.0.1', 41714)|0.5|47.8|59.3|544.78565120697|||||2026-03-06 02:01:55
10|('127.0.0.1', 41722)|1.0|47.8|59.3|550.910573482513|||||2026-03-06 02:02:01
11|('127.0.0.1', 60264)|0.0|47.8|59.3|556.960312366486|||||2026-03-06 02:02:07
12|('127.0.0.1', 60276)|1.1|47.8|59.3|563.002655506134|||||2026-03-06 02:02:14
13|('127.0.0.1', 60586)|0.5|48.0|59.3|569.057591915131|||||2026-03-06 02:02:20
14|('127.0.0.1', 57222)|2.1|48.0|59.3|575.115680217743|||||2026-03-06 02:02:26
15|('127.0.0.1', 57226)|0.5|48.0|59.3|581.145050287247|||||2026-03-06 02:02:32
16|('127.0.0.1', 44690)|0.5|48.0|59.3|587.209727525711|||||2026-03-06 02:02:38
```

LIMITES ET AMELIORATIONS

Le système développé permet de supervisé plusieurs machines et de collecter différentes métriques systèmes. Cependant, certaines améliorations pourraient être apportées :

. Ajout d'une interface graphique pour visualiser les métriques en temps réel

. Ajout d'un système d'alertes avancé

. Support d'un plus grand nombre de services et de ports

. Amélioration de la sécurité des communications

CONCLUSION

Ce projet a permis de mettre en œuvre un système de supervision distribué basé sur une architecture client-serveur.

Les agents de supervision collectent périodiquement les métriques systèmes et les transmettent au serveur central via une connexion TCP. Le serveur analyse ces informations et les enregistre dans une base de données afin de permettre leur consultation.

Les résultats obtenus montrent que le système fonctionne correctement et permet de superviser efficacement l'état des machines.

Bibliographie/Webographie

1) Python Software Foundation

   Documentation officielle de Python.

   https://docs.python.org

2) Documentation officielle de la bibliothèque psutil.

   https://psutil.readthedocs.io
   SQLite Documentation.SQLite
   Database Engine.
   https://www.sqlite.org/docs.html

2) TutorialsPoint.PythonSocket
   Programming Tutorial.
   https://www.tutorialspoint.com/python/python_networking.htm

3) Geeksforgeeks.Multithreading in Python.
   https://www.geeksforgeeks.org/multithreading-python-set-1

4) Diagrams.net.Draw.io-Diagram Tool.
   https://app.diagrams.net

LIEN DU GIT : https://github.com/Sophiendiaye97/Projet-Systeme-Reparti